

30. Substring with Concatenation of All Words — Complete Intuition & Explanation (Notes)

1. Problem Statement

You are given:

- A string `s`
- An array `words` where every word has the same length.

You have to find all starting indices where the substring is exactly the concatenation of all the words in any order.

Example

```
s = "barfoofoobarthefoobarman"
words = ["bar", "foo", "the"]
```

Possible valid concatenations are

```
barfoothe
barthefoo
foobarthe
thefoobar
...
```

If any of these appears in `s`, return its starting index.

2. What is the first observation?

Every word has equal length.

Suppose

```
words = ["bar", "foo", "the"]
```

- Each word length = 3
- Number of words = 3

Therefore every valid substring must have length

$$3 \times 3 = 9$$

So we are not searching random length substrings.

Every candidate window is exactly

```
windowLength = wordLength × numberOfWords
```

This is the first observation.

3. What makes a window valid?

Suppose

```
window = "foobarthe"
```

Split into words

foo

bar

the

Frequency

foo ->1

bar ->1

the ->1

Required frequency

foo ->1

bar ->1

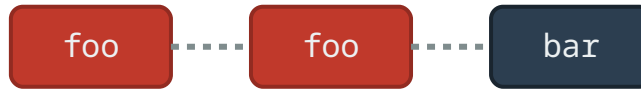
the ->1

They are identical. So the window is valid.

Suppose

```
Window = "foofoobar"
```

Split



Frequency

foo ->2
bar ->1

Required

foo ->1
bar ->1
the ->1

Not equal.

Invalid.

Therefore

We only need to compare frequency of words, not the entire substring.

Hence,

```
unordered_map<string,int> need;
```

stores the required frequency.

4. Why do we use Sliding Window?

Suppose

```
wordLength = 3
```

Window

```
bar foo the
```

Move one character

```
arf oot he...
```

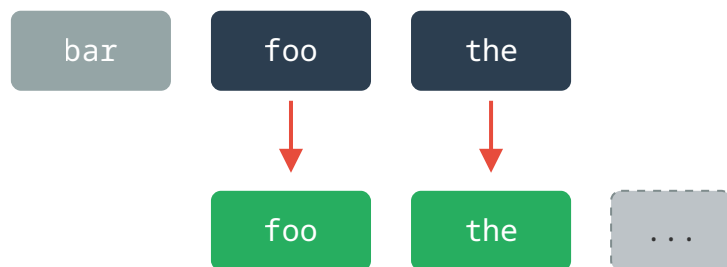
Now every word becomes broken.

The chunks are meaningless.

So normal sliding window doesn't work.

Instead,

move one complete word every time.



Notice

```
left += wordLength  
right += wordLength
```

NOT

```
left++  
right++
```

This is the biggest observation.

5. Why do we need a HashMap?

Suppose

```
words
```

```
foo
```

```
bar
```

```
foo
```

Store

```
need
```

```
foo ->2
```

```
bar ->1
```

Now while scanning,
maintain another hashmap

```
window
```

which stores

```
Current window frequencies
```

Example

Current window

```
foo
```

```
bar
```

Window map

```
foo ->1
```

```
bar ->1
```

Whenever

```
window == need
```

we have found one answer.

6. Why can't we simply scan from index 0?

This is the most confusing part.

Let's understand carefully.

Suppose

```
wordLength = 3
```

String

```
barfoofoobar
```

Index

```
0 1 2 3 4 5 6 7 8 9 ...
```

Starting from index 0

Words become

```
bar | foo | foo | bar
```

Perfect.

Starting from index 1

Words become

```
arf | oof | oob | ...
```

Different alignment.

Starting from index 2

Words become

```
rfo | ofo | oba | ...
```

Again different alignment.

These are the only three possible alignments.

7. Why does offset only go until wordLength-1?

This is the most important observation.

Suppose

```
wordLength = 3
```

Offsets are

Offset 0

```
0 3 6 9 12 ...
```

Offset 1

```
1 4 7 10 ...
```

Offset 2

```
2 5 8 11 ...
```

Now suppose you start with

```
offset = 3
```

Positions become

```
3 6 9 12 ...
```

Look carefully.

These positions were already visited by

```
offset = 0
```

because

```
0 → 3 → 6 → 9 → 12
```

Exactly the same sequence.

Nothing new.

Similarly,

Offset 4

```
4 7 10 13 ...
```

Already covered by

```
offset = 1
```

Offset 5

Already covered by

```
offset = 2
```

Therefore

After

```
offset = wordLength - 1
```

every alignment starts repeating.

There is absolutely no benefit checking larger offsets.

Mathematical View

Every index belongs to one remainder class.

Suppose

```
wordLength = 3
```

Index

```
0 1 2 3 4 5 6 7 8 9
```

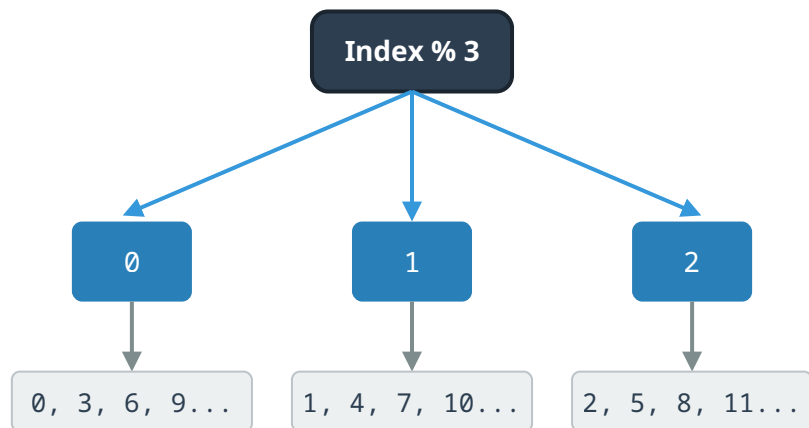
Take modulo

```
0%3 =0
1%3 =1
2%3 =2
3%3 =0
4%3 =1
5%3 =2
6%3 =0
```

Only

```
0
1
2
```

are possible.



Exactly

```
for(int offset=0; offset<wordLength; offset++)
```

Every starting index belongs to one of these groups.
Hence checking more offsets would only repeat work.

8. Sliding Window Process

For every offset,
maintain

```
left  
right  
window map  
count
```

Initially

```
window = {}  
count = 0
```

Move

```
right += wordLength
```

Take next word.

Case 1 : Word not present

Example

```
Need  
  
bar  
foo  
the
```

Current word

```
cat
```

Since

```
cat
```

can never be part of any valid window,
reset everything.

```
window.clear()  
count=0  
left = right + wordLength
```

Start fresh.

Case 2 : Frequency becomes too much

Need

```
foo ->1
```

Current

```
foo ->2
```

Window is invalid.

Remove words from left until

```
foo ->1
```

Again valid.

Case 3 : Window contains all words

Suppose

```
count =3
```

Required

```
wordCount=3
```

Window

```
foo  
bar  
the
```

Frequency matches.

Store

```
left
```

Then remove the leftmost word and continue searching because another valid window may overlap.

9. Dry Run

Example

```
s  
  
bar foo foo bar the foo bar man  
  
words  
  
bar  
foo  
the
```

Need

```
bar ->1  
  
foo ->1  
  
the ->1
```

right =0

Word

bar

Window

bar ->1

count

1

right =3

Word

foo

Window

bar ->1

foo ->1

count

2

right =6

Word

foo

Window

foo ->2

Too many.

Shrink.

Remove

bar

Still

foo ->2

Shrink again.

Remove

foo

Now

foo ->1

Window becomes valid.

right =9

Add

bar

Window

foo ->1

bar ->1

right =12

Add

the

Window

foo ->1

bar ->1

the ->1

count

3

Perfect.

Answer

6

Remove left.

Continue.

right =15

Add

foo

Again

foo

bar

the

Answer

9

Continue.

`right = 18`

Add

`bar`

Again valid.

Answer

`12`

Final answer

`[6, 9, 12]`

10. Time Complexity

Suppose

`N = length of string`

`L = word length`

We process

`L`

different offsets.

For each offset,

every word enters and leaves the sliding window at most once.

Hence

`Time = O(N)`

(or more precisely $O(N \times L)$, which is effectively linear because L is fixed for a given input).

Space

`O(number of distinct words)`

for the two hash maps.

11. Complete Intuition (Interview Answer)

Since every word has the same length, every valid substring has a fixed length equal to $\text{wordLength} \times \text{numberOfWords}$. Instead of moving character by character, I treat the string as blocks of wordLength characters. There are only wordLength possible block alignments (offsets 0 to $\text{wordLength}-1$). For each alignment, I use a sliding window that moves one word at a time. A hashmap need stores the required frequency of each word, while another hashmap window stores the frequencies inside the current window. If I encounter an invalid word, I reset the window. If a word appears more times than required, I shrink the window from the left until the frequency becomes valid again. Whenever the window contains exactly numberOfWords with matching frequencies, I record the starting index. This gives an efficient solution by scanning each alignment only once.